TECHNICAL MANUAL

Ghaf tree-crown mapping from UAV imagery

Delineating *Prosopis cineraria* crowns in area-wide UAV orthomosaics

---

FastViT-MA36 with a Mask2Former decode head, and five alternatives.
Six trained models, 8 641 labelled tiles, and an inference path that maps a whole survey.

Repository **brakuta/Ghaf-Tree-Crown-Mapping-from-UAV-Data**
Documented at commit **54ccf74**
Values are traceable to **docs/handover/FACTS.yml**, which wins wherever it and this manual could disagree.
First issue, September 2026

Contents

You can stop reading here

Chapters 3, 4 and 10 are the whole job if what you need is a canopy map from an orthomosaic: install the environment, prove it, run the model. Chapter 15 is the error catalogue and the only other chapter worth reading before something goes wrong. Everything between exists for the reader who has to change something.

The chapters run in the order the work is done: the data, then what the configuration will do with it, then training, scoring and prediction. Read chapter 2 when you have to find a file, and chapters 5 and 6 before building a dataset of your own — the mask encoding is the part that fails silently, and chapter 6 says which settings can be changed without invalidating the published scores. Skip chapter 7 and chapter 13 unless you are training: six trained models are supplied, and reproducing one of them costs a day of GPU time. Chapter 14 is for whoever has to hand the system on again.

CHAPTER 1

What this system does, and what you were given

The system takes a UAV orthomosaic and returns the Ghaf crowns in it as GIS polygons. This chapter says what the models were trained to do, what arrived with them, and what the system cannot do — the last of which is the part worth reading twice.

1.1 The task, exactly

Two classes and nothing else: `0` for background and `1` for a Ghaf crown. The models are semantic segmentors, so they label pixels, not objects; the crown polygons in the output are traced from the labelled pixels afterwards. Training tiles are 1024 × 1024 pixel PNG pairs, an image and a mask sharing a filename stem, and the inference window is the same 1024 pixels for the same reason.

A mask is a label image, not a picture. Opened in an image viewer it is almost black, because its two values are 0 and 1 out of a possible 255. That is correct, and it is the first thing that makes a newcomer think the data is broken.

Property	Value	Where it is fixed
Classes	`background`, `ghaf`	`ghaf/datasets.py`
Mask encoding	0 background, 1 ghaf, single band	`ghaf/datasets.py`

Property	Value	Where it is fixed
`reduce_zero_label`	`False` — background is supervised, not ignored	`ghaf/datasets.py`
Tile and crop size	1024 × 1024 px	`configs/_base_/ghaf.py`
Tile format	PNG, image and mask sharing a stem	`ghaf/datasets.py`
Coordinate system	EPSG:32640 (UTM zone 40 N)	the imagery itself

The dataset contract. Changing any row of it invalidates every published score.

1.2 The six models

All six share one dataset, one augmentation pipeline, one schedule and one metric implementation, so the differences between them are attributable to the encoder and the decode head. Scores are on the held-out `testing/ghaf26` split. Use FastViT-MA36 unless you have a reason not to; the other five are the comparison the paper reports, and every procedure in this manual works with any of them by substituting two paths.

Model key	Backbone	Head	Params	mIoU	F1
`fastvit-ma36_mask2former`	FastViT-MA36	Mask2Former	62,549,115	**79.32**	**87.22**
`poolformer-s36_fpn`	PoolFormer-S36	FPNHead	34,600,137	78.65	86.72
`dpn98_fpn`	DPN-98	FPNHead	65,346,639	78.19	86.35
`convnext-small_upernet`	ConvNeXt-S	UPerHead	81,776,049	78.02	86.20
`resnet-50_mask2former`	ResNet-50	Mask2Former	44,056,504	77.69	85.98
`efficientnet-b3_fpn`	EfficientNet-B3	FPNHead	13,734,524	70.77	80.29

Parameter counts are exact sums over each `state_dict`, measured from the checkpoint rather than transcribed (`ghaf/release.py`).

Only FastViT-MA36 has a recorded validation score, 80.14 mIoU and 87.87 F1. The validation figures for the other five were never established, and this manual does not estimate them.

1.3 What arrived, and what did not

The code is public. The weights, the imagery and the labelled tiles are not: they travel separately, and the bundle below is what a recipient is sent. Sizes are those of the bundle as it was built, from commit `71a07db`.

Part	Size	Contents
`code/`	63 files	Exactly what `git ls-files` reported at that commit. No working files, no local checkpoints
`models/`	2.22 GB	Six folders, each a resolved configuration beside its weights and a `metadata.json`
`init-weights/`	0.90 GB	ImageNet initialisation weights for the six backbones. Needed only to train without internet access; not used for inference
`data/ghaf/`	15.64 GB	19,583 files. The three named splits only — the working tree it was cut from held 49,490 files more
`samples/`	0.17 GB	Two files: the 8192 × 8192 clip and its overview. **The full orthomosaic is not in the bundle**
`predictions/testing/`	768 files	767 predicted masks for the test split, and one `summary.json`
`MANIFEST.json`	—	Every part, its size, its file count, and the commit the code came from. This is how a result is traced back to a version

The handover bundle, 18.93 GB in total.

1.4 What it cannot do

Four limits, stated because each one has cost somebody a day.

- **Two classes.** The models separate Ghaf from not-Ghaf. They do not distinguish one tree species from another, and a second species in the imagery will be labelled background or crown according to how much it resembles a Ghaf, with no signal that it happened.

- **Touching crowns merge.** Semantic segmentation labels pixels, so two crowns whose canopies overlap are one connected region and become one polygon. The crown count is therefore a lower bound in dense stands. It was never quantified on this imagery.
- **8-bit RGB only.** The pipeline reads three bands as red, green and blue. Other band orders need `--bands`; 16-bit input is rejected outright, which at least fails loudly.
- **Runs are not reproducible.** `randomness` is commented out in `configs/_base_/ghaf.py`, so no seed is set. Two training runs of the same configuration will not agree exactly, and the seed mmengine chose exists only in the run log.

> **THE FULL SURVEY MOSAIC HAS NEVER BEEN RUN END TO END**
>
> Every timing in this manual for area-wide inference comes from the 8192 × 8192 clip. The full `Kalba26.tif` is 84,072 × 103,691 pixels, which is 8.7 billion; the 78.5 GB of scratch space and the 33,128 windows quoted in chapter 5 are arithmetic on that size, not a measurement. Nobody has watched it finish, and nothing in the pipeline reports a partial result — a run that dies at window 30,000 leaves scratch files and no output.

Checklist

- You know which of the six models you are using, and why.
- You know that a mask looks black and that this is correct.
- You have `MANIFEST.json`, so any result can be traced to a commit.
- You have read the four limits above before promising anybody a tree count.

CHAPTER

2 The repository, folder by folder

Fifty-four files in four directories — `git ls-files ghaf configs tools tests` counts them. This chapter is the map: what each file is for, and which of them you will ever open. Read it when you need to find something, not before.

2.1 `ghaf/` — the library

The importable package. Nothing here is run directly except `ghaf.inference.large_image`, which is a module rather than a script in `tools/` because it is part of the library that other code calls.

File	What it holds
`config.py`	Points a configuration at a dataset root, and disables ImageNet initialisation when a checkpoint is being loaded anyway
`datasets.py`	`GhafDataset` — the two-class tile dataset, and the guard that refuses `reduce_zero_label=True`
`splits.py`	Where each split lives, declared once. `check_dataset`, `predict_split` and `build_handover` all read it, so the layout cannot drift between them
`environment.py`	Confirms the framework is installed in the interpreter that is running, and quietens the repeated framework warnings
`init_weights.py`	Finds the ImageNet initialisation weights that ship with the bundle
`paths.py`	One order for a folder listing on every operating system. Windows sorts file names without regard to case and Linux does not, so sorting the platform's way would give `--limit` a different set of images on each machine
`release.py`	The published models: digests, byte sizes, parameter counts and scores. The single source of truth that `smoke_test` and `export_release` both check against
`models/fastvit.py`, `models/dpn.py`	The two backbones that mmpretrain does not supply. `models/modules/` holds the MobileOne and RepLKNet blocks FastViT is built from
`inference/tiling.py`	Window planning and Gaussian blending, in pure NumPy. No mmseg, no torch, no rasterio — which is why the geometry can be tested without a GPU

File	What it holds
`inference/large_image.py`	Orthomosaic inference: windowed reads, memory-mapped accumulators, georeferenced output

2.2 `configs/` — the recipes

`_base_/ghaf.py` holds the dataset, the augmentation pipeline, the schedule and the runtime. The six files in `configs/ghaf/` inherit it and differ only in backbone, neck and decode head. That is what makes the comparison in chapter 1 a comparison of architectures rather than of training budgets.

> **EDITING A BASE CONFIG CHANGES RESULTS ALREADY PUBLISHED**
> Every one of the six models inherits `configs/_base_/ghaf.py`. A change to the crop size, the evaluator or the schedule in that file silently redefines what the published scores mean, and nothing in the pipeline will warn you. Copy it to a new file and edit the copy.

2.3 `tools/` — the programs you run

Program	What it does	Chapter
`check_dataset.py`	Verifies pairing, sizes and mask values before a tree is used	4
`smoke_test.py`	Builds all six models, verifies each checkpoint against its published digest, runs one prediction through each	4
`train.py`	Trains a model, or fine-tunes one	7, 13
`test.py`	Scores a model against a labelled split	8
`predict_split.py`	One predicted mask per tile for a labelled split	9
`predict_folder.py`	Maps every image in a folder, one GeoPackage of crowns per image	11
`make_sample.py`	Cuts a small georeferenced clip out of a large mosaic	5
`fetch_init_weights.py`	Collects the ImageNet weights once, while online	7
`export_release.py`	Assembles the models folder for sharing, verifying every checkpoint before and after the copy	14
`build_handover.py`	Assembles the whole bundle described in chapter 1	14

Listed in the order the manual runs them. `ghaf.inference.large_image` *is the one that maps a whole orthomosaic (chapter 10); it is a module rather than a script here.*

2.4 `tests/` and `docs/`

Eighteen test files and 345 tests. They run without a GPU, without mmcv and without the dataset, which is what makes them worth running on a machine that has just been set up: a pass proves the code is intact even before the weights arrive. `docs/` holds this manual and its sources, the quick reference, and four reference documents inherited from the repository.

2.5 Where results are written

No program writes into `data/`, `models/` or `code/`. Every one of them takes an output path on the command line, and the examples throughout this manual write to `output/`. Training is the exception worth knowing: `train.py` writes to `work_dirs/<config-name>/` relative to the working directory, and that is where the loss curves and the seed of a run survive.

Checklist

- You can name the file that defines the dataset layout.
- You know that the six configs differ only in architecture.
- You know not to edit anything under `configs/_base_/`.
- You know where a training run leaves its logs.

CHAPTER

3 Installing the environment

Twenty minutes, once per machine, and the version numbers are not suggestions. This chapter installs the stack the six models were trained against; chapter 4 proves the installation before anything depends on it.

3.1 What the machine needs

Item	Requirement	Notes
Operating system	Windows 10 or 11, macOS, or Linux	Commands here are for the Windows Command Prompt. On macOS and Linux the only difference is `/` for `\`
GPU	NVIDIA, 8 GB or more	The workstation the models were trained on is an RTX A5000. Without a GPU every procedure still runs with `--device cpu`, far more slowly; the factor was never measured
Disk, bundle	20 GB	Code, models, initialisation weights, tiles and the sample clip
Disk, one run	9 bytes per pixel of the image	The 8192 × 8192 clip needs 0.6 GB. The full mosaic needs 78.5 GB. Released when the run ends, and checked before it starts
Python	3.9	Installed by conda below. No system Python is involved
Prerequisite	Miniconda or Anaconda	From docs.conda.io

System requirements.

3.2 Conventions in this manual

A shaded block is a command. Enter it, press Enter, and wait for the prompt before the next one. A caret at the end of a line continues that command onto the next in the Command Prompt; the continuation starts hard against the left margin on purpose, because leading spaces would otherwise be carried into the argument. On macOS or Linux the continuation character is a backslash instead.

The paths are the ones on the machine this was written on. `D:\ghaf-project\` is where the bundle was unpacked there; substitute wherever it was unpacked here, and every command works unchanged. No program looks for that name, so `no such folder` or `no checkpoint under` naming `D:\ghaf-project` is the example path being run as though it were real, not a broken installation.

> **PASTE ONE COMMAND AT A TIME**
> Several terminals join a multi-line paste into a single line and run something nobody typed. It usually fails loudly. It does not always: a joined `cd` followed by a `python` call can run the right program in the wrong folder, which produces a result rather than an error.

3.3 Step 1 — create the environment

```
conda create -n ghaf python=3.9 -y
conda activate ghaf
```

The prompt now begins with `(ghaf)`. Every command in this manual assumes it. A new terminal window opens outside the environment, so this is run again in each new window; forgetting it is the first entry in the error catalogue for a reason.

3.4 Step 2 — PyTorch

The version the released models were trained with. About 2 GB.

```
python -m pip install torch==1.12.1+cu113 torchvision==0.13.1+cu113 ^
--extra-index-url https://download.pytorch.org/whl/cu113
```

Without an NVIDIA GPU, install the CPU build instead.

```
python -m pip install torch==1.12.1 torchvision==0.13.1
```

3.5 Step 3 — the OpenMMLab stack

In this order. `mmcv` must stay below 2.2.0, which mmsegmentation 1.2.2 asserts at import time, and `mim` fetches the prebuilt wheel matched to the installed PyTorch and CUDA rather than compiling it.

```
python -m pip install -U openmim
python -m mim install mmengine==0.10.7 "mmcv>=2.0.0rc4,<2.2.0"
python -m pip install mmsegmentation==1.2.2 mmdet==3.3.0 mmpretrain==1.2.0
```

`mmdet` supplies the Mask2Former head and `mmpretrain` the ConvNeXt, PoolFormer and EfficientNet backbones. The second command is the slow one; several minutes is normal.

3.6 Step 4 — this project

```
cd /d D:\ghaf-project\code
python -m pip install -r requirements.txt
python -m pip install -e ".[test]"
```

`requirements.txt` adds `timm` 1.0.19 for the FastViT blocks and DPN-98, `rasterio` and `geopandas` for the georeferenced input and output, and `ftfy` and `regex`, which mmsegmentation 1.2.2 needs to import at all — its own metadata does not say so. NumPy is held below 2.0 because PyTorch 1.12.1 is built against the NumPy 1.x binary interface.

> **WHAT THE LAST STEP PRINTS, AND WHAT NONE OF IT MEANS**
>
> pip may report that `opencv-python` requires `numpy>=2`, and may then replace a newer OpenCV with 4.11.0.86 — a line reading `Attempting uninstall: opencv-python`. Both are the NumPy pin doing its work: 4.11.0.86 is the newest OpenCV built against NumPy 1.x, and it is the only kind this project can use. Nothing here imports OpenCV directly; mmengine reads images with it and is content with any version from 3 upwards. pip may also list conflicts among packages this project does not import. Only a line beginning `ERROR:` that stops the install matters.

3.7 Paths with spaces, and paths with an ampersand

Quote any path containing a space. Quote a path containing `&` without exception: the Command Prompt reads `&` as the end of one command and the start of another, so an unquoted path fails with "The system cannot find the path specified" printed twice, once for each half.

```
cd /d "Z:\Survey Data\Cineraria_Data & Model\ghaf-project\code"
```

Checklist

- The prompt shows `(ghaf)`.
- `python -c "import sys; print(sys.executable)"` prints a path containing `envs\ghaf`.
- No line beginning `ERROR:` stopped any install step.
- You are in `code\`, and chapter 4 is next.

CHAPTER

4 Verifying the installation

Three checks, five minutes. They establish that the code, the weights and the tiles arrived intact and agree with one another. Run them after installing, and again on any day when a result looks wrong.

4.1 Check 1 — the code

No GPU, no weights and no dataset are needed, which is what makes this worth running on a machine that has only just been set up.

```
python -m pytest tests\ -q
```

```
.....................................................
345 passed, 1 skipped in 28.69s
```

Any tally of passes with no `F` in the progress output is a pass. The number skipped is not fixed and does not need to match: a test that needs `geopandas`, `torch` or `git` skips itself where they are absent, so a machine without them reports more skips than this one, and is not failing. `No module named mmengine` here means a different Python is running than the one the packages went into; chapter 15 has the remedy.

4.2 Check 2 — the weights

This builds all six models from their configurations, compares each checkpoint against its published SHA-256, loads the weights into the model, names any tensor the two do not share, and runs one prediction through each. About 90 seconds on a GPU.

```
python tools\smoke_test.py --checkpoints D:\ghaf-project\models
```

```
model                          digest             weights  prediction
-----------------------------------------------------------------------
fastvit-ma36_mask2former          ok    all 1,558 matched  0.00% ghaf
poolformer-s36_fpn                ok      all 436 matched  0.00% ghaf
dpn98_fpn                         ok      all 676 matched  0.00% ghaf
convnext-small_upernet            ok      all 430 matched  0.00% ghaf
resnet-50_mask2former             ok      all 610 matched  0.00% ghaf
efficientnet-b3_fpn               ok      all 610 matched  0.00% ghaf

all 6 model(s) built and ran a forward pass
```

Column	What it establishes
`digest` reads `ok`	The file is byte-for-byte the one that was released. Nothing was corrupted in transit or in the copy
`all N matched`	Every tensor in the checkpoint found its place in the model built from the configuration. Nothing missing, nothing left over
`+0` in the delta column	The parameter count equals the published figure exactly
`0.00% ghaf`	**Expected.** The forward pass runs on a blank synthetic tile, not on imagery, so finding no trees is the correct answer here

SCREENS OF USERWARNING ARE NOT A FAULT

Every command that loads a model prints warnings from inside PyTorch and mmsegmentation about `__floordiv__`, `torch.meshgrid`, binary segmentation and `build_loss`. They appear on a correct run. The result is the table printed after them, not the absence of warnings.

`--only fastvit-ma36_mask2former` limits the check to one model, and `--strict` turns a tensor-count mismatch from a report into a non-zero exit, which is what a scripted check wants.

4.3 Check 3 — the tiles

```
python tools\check_dataset.py D:\ghaf-project\data\ghaf
```

```
split          images    masks   paired  checked  status
-------------------------------------------------------
training         7005     7005     7005      200  ok
validation        869      869      869      200  ok
testing           767      767      767      200  ok

8,641 paired tile(s) across 3 split(s)
dataset looks usable
```

Variant	What it does	When
`--sample 0`	Pairing only. Opens no file at all	Seconds, even over a network drive. Use it first after any copy
`--sample 25`	Opens 25 tiles per split	A minute. Enough to catch a truncated copy
`--full`	Opens all 8,641 tiles	Slow. Worth doing once, on the machine that will train
`--json report.json`	Writes the same result as JSON	When the check runs unattended

Checklist

- The test suite passed.
- All six digests read `ok`, and every tensor matched.
- You understand why the prediction column reads `0.00% ghaf`.
- The three splits report 7005, 869 and 767 pairs.

CHAPTER

5 The data

What the tiles are, how they are laid out, and the two properties that break a dataset silently. Read this before building a dataset of your own; skip it if you are only running the supplied models.

5.1 Layout

Three splits, declared once in `ghaf/splits.py` and read from there by `check_dataset`, `predict_split` and `build_handover` alike, so the layout cannot drift between the programs that depend on it.

```
data\ghaf\
  training\images\      7005 PNG tiles, 1024 x 1024, 8-bit RGB
  training\masks\       7005 PNG masks, one per image
  validation\images\     869 tiles
  validation\masks\      869 masks
  testing\ghaf26\images\  767 tiles, with .pgw and .png.aux.xml
  testing\ghaf26\masks\   767 masks
```

Split	Pairs	What it is for
`training`	7,005	Fitting the model
`validation`	869	Scored every 3,500 iterations during training; the best checkpoint is chosen on it
`testing/ghaf26`	767	Held out entirely. The source of every published score

8,641 pairs in total, counted by `check_dataset.py`.

5.2 The two things that fail silently

Neither raises anything. Both produce a model that trains to a respectable number and predicts nothing, which is why they are the first thing to check when a result looks impossible.

> **A MASK WITH THE WRONG VALUES TRAINS WITHOUT COMPLAINT**
>
> The mask pixel value **is** the class index: 0 for background, 1 for ghaf. A mask exported as 0 and 255, which is what most annotation tools produce by default, is not rejected — mmseg reads 255 as class 255, the loss ignores it, and the model learns that nothing is a crown. The loss curve looks unremarkable. `check_dataset.py` is the only thing that catches this, and it is why it exists.
>
> `reduce_zero_label` must stay `False`. With it set, background is ignored rather than supervised, which silently converts the task into something the published scores no longer describe. `GhafDataset` refuses the argument rather than accepting it.

5.3 Georeferencing, and which tiles carry it

The test tiles have `.pgw`, `.png.aux.xml` and `.ovr` companion files beside them. A PNG cannot hold a coordinate system, so the world file carries the transform and GDAL keeps the CRS in the `.aux.xml`. That is why a prediction made from a test tile opens in QGIS in the right place, and a prediction from a training tile does not. The training and validation tiles have no companion files, which affects neither training nor scoring. Keep the companions beside the tiles they belong to; moving the PNG alone silently strips the position.

5.4 Building a dataset of your own

Match the layout above: 1024 × 1024 PNG pairs sharing a stem, masks containing only 0 and 1, one folder per split. Then verify it before training rather than after.

```
python tools\check_dataset.py D:\ghaf-project\data\new-site --full
```

A fault reported here costs a minute. The same fault found after a training run costs the run, and it will not be obvious which fault it was, because a model trained on empty masks converges to predicting nothing and reports a high mIoU while doing it — the background class alone is 96 per cent of the pixels.

Checklist

- Every image has a mask with the same stem.
- Masks contain only 0 and 1, verified rather than assumed.
- Companion files travelled with the tiles they belong to.
- `check_dataset.py` reports `dataset looks usable`.

CHAPTER

6 The configuration files

Every number the training and scoring commands obey lives in two files: one shared by all six models, and one per model. This chapter reads the shared file field by field, says which fields are safe to change and which invalidate the published scores, and gives the rule that keeps a change from reaching the six models at once.

6.1 Two files, and what each one owns

A model configuration is assembled from `configs/_base_/ghaf.py` and the file that names it. The model file begins `_base_ = ['../_base_/ghaf.py']`, which reads the shared file first; anything the model file then defines replaces what it found. That division is what makes the comparison in chapter 1 a comparison of architectures: the six models differ only in the half they own.

File	Owns	Fields
`configs/_base_/ghaf.py`	Everything the six models share	dataset, pipelines, preprocessor, the three dataloaders, the evaluator, the schedule, the hooks, the runtime

File	Owns	Fields
`configs/ghaf/<model>.py`	The architecture, and how it is optimised	`model` (backbone, neck, decode head, losses) and `optim_wrapper` (the optimiser, its learning rate, and the per-parameter multipliers)

The optimiser sits with the model, not in the shared file: a Mask2Former head and an FPN head do not want the same learning rate.

> **NEVER EDIT THE SHARED FILE IN PLACE**
> All six configurations inherit it. A change to the crop size, the evaluator or the schedule there silently re-defines what every published score means, and nothing in the pipeline warns you. To try something different, copy the file, change the copy, and point a new model config at it. Section 6.8 does exactly that.

6.2 The dataset block

```
dataset_type = 'GhafDataset'
data_root    = 'data/ghaf'
crop_size    = (1024, 1024)
```

Field	Value	What it decides
`dataset_type`	`GhafDataset`	The two-class reader in `ghaf/datasets.py`. It is what refuses `reduce_zero_label=True`
`data_root`	`data/ghaf`	**Relative**, and read while the file is parsed. Resolved against the directory the command is typed in, which is `code\`. Override it with `--data-root`, never with `--cfg-options`
`crop_size`	1024 × 1024	The size the preprocessor pads to, and the size the models were trained at. Changing it invalidates every published score

`--cfg-options data_root=...` is accepted and does nothing useful: `data_root` is a plain module-level variable, copied into each dataloader as the file is read, so setting the key afterwards changes something nothing looks at. `ghaf/config.py` makes the substitution the way it has to be made, once per dataset, which is what `--data-root` calls.

6.3 The two pipelines

A pipeline is the list of transforms applied to a tile between the file and the model. There are two, and the whole difference between them is one line.

```
train_pipeline = [
    dict(type='LoadImageFromFile'),
    dict(type='LoadAnnotations', reduce_zero_label=False),
    dict(type='RandomFlip', prob=0.5),
    dict(type='PackSegInputs'),
]

test_pipeline = [                       # the same, without the flip
    dict(type='LoadImageFromFile'),
    dict(type='LoadAnnotations', reduce_zero_label=False),
    dict(type='PackSegInputs'),
]
```

Transform	What it does
`LoadImageFromFile`	Reads the tile as 8-bit BGR
`LoadAnnotations`	Reads the mask as class indices. `reduce_zero_label=False` keeps background as a supervised class rather than an ignored one — the setting chapter 5 says must not change
`RandomFlip`	Horizontal flip, half the time. **The only augmentation in the study**: no scale jitter, no random crop, no colour distortion. Tiles are fed at native resolution
`PackSegInputs`	Packs the image and mask into the structure the model expects

Adding augmentation is a legitimate experiment and a change to what the numbers mean. Do it in a copy of the file, and score the result against the same test split before believing it.

6.4 The preprocessor

What happens to a tile between the pipeline and the model: normalisation, channel order, and the padding that makes an awkward size fit.

```
data_preprocessor = dict(
    type='SegDataPreProcessor',
    mean=[123.675, 116.28, 103.53],     # ImageNet statistics
    std=[58.395, 57.12, 57.375],
    bgr_to_rgb=True,
    pad_val=0, seg_pad_val=255,
    size=crop_size,
    test_cfg=dict(size_divisor=32))
```

Normalisation uses ImageNet statistics because every backbone was pre-trained on ImageNet; `bgr_to_rgb` undoes the channel order the loader produced. `seg_pad_val=255` matters: padding in the mask is labelled 255, which the loss ignores, so a padded edge contributes nothing to the gradient. `size_divisor=32` rounds an odd-sized test image up to something the strides divide.

6.5 The three dataloaders

One per split, identical but for the folder, the sampler and the batch size. Each names its folders relative to `data_root`, which is why `--data-root` moves all three together.

	`train_dataloader`	`val_dataloader`	`test_dataloader`
`img_path`	`training/images`	`validation/images`	`testing/ghaf26/images`
`seg_map_path`	`training/masks`	`validation/masks`	`testing/ghaf26/masks`
`pipeline`	`train_pipeline`	`test_pipeline`	`test_pipeline`
`batch_size`	2	1	1
`num_workers`	2	2	2
`sampler`	`InfiniteSampler`, shuffled	`DefaultSampler`	`DefaultSampler`

`InfiniteSampler` *is what an iteration-based schedule needs: it never runs out, so training stops at an iteration count rather than at the end of an epoch.*

`batch_size=2` is what the workstation in chapter 3 fitted at 1024 pixels; it is a memory limit, not a tuned value. Raising it changes the effective batch and therefore the result. Scoring uses 1 so that no tile is padded to match another. `num_workers=2` is the number of loader processes, and is safe to change: it affects speed only.

6.6 The schedule

```
train_cfg = dict(type='IterBasedTrainLoop',
                 max_iters=160000, val_interval=3500)
param_scheduler = [dict(type='PolyLR', eta_min=0, power=0.9,
                        begin=0, end=160000, by_epoch=False)]
```

Field	Value	Meaning
`max_iters`	160,000	The length of the schedule. Not the length of a run: the released checkpoints stopped far earlier, at 3,500 to 38,500 iterations
`val_interval`	3,500	Score the validation split this often. The checkpoint hook uses the same interval, which is why the released filenames carry those numbers
`PolyLR`, `power=0.9`	—	Polynomial decay of the learning rate from its initial value to `eta_min`

Field	Value	Meaning
`end=160000`	—	**Must match** `max_iters`**.** The decay is computed against `end`, so shortening the schedule without changing both leaves the rate decaying on the old curve

The optimiser is in the model file, not here. For FastViT-MA36 it is AdamW at a learning rate of 1e-4 with weight decay 0.05, gradients clipped at a norm of 0.01, and a `paramwise_cfg` that gives the backbone a tenth of the learning rate and exempts every normalisation layer from decay.

6.7 The hooks, and what each leaves on disk

A hook is a callback the runner fires at a fixed point — every iteration, every validation, the end of a run. These six are on by default, and between them they account for everything a training run writes.

Hook	Setting	What you see
`LoggerHook`	`interval=50`	A line every 50 iterations, with loss and learning rate, to the console and to `work_dirs/<config>/<timestamp>/`
`CheckpointHook`	`interval=3500`, `save_best='mIoU'`	`iter_3500.pth` and so on, plus `best_mIoU_iter_N.pth`, which is rewritten whenever the validation mIoU improves
`ParamSchedulerHook`	—	Steps `PolyLR`. Nothing on disk
`IterTimerHook`	—	The `time:` and `eta:` fields in the log
`DistSamplerSeedHook`	—	Reseeds the sampler each epoch under distributed training
`SegVisualizationHook`	`draw=False`	Nothing, until `--show-dir` turns it on. It says so at startup, and that warning is not a fault

`best_mIoU_iter_N.pth` is the file the released models are: the checkpoint that scored best on the validation split, not the last one written. Two hooks together decide which iterations can be chosen — `val_interval` and the checkpoint interval — and both are 3,500.

6.8 The runtime block

Field	Value	Why it is there
`custom_imports`	`ghaf.datasets`, `ghaf.models`	**Load-bearing.** Without it `GhafDataset`, `FastViTMA36` and `DPN98` are not in the registry and the config fails to build
`default_scope`	`mmseg`	Which registry a bare type name is looked up in. Types written `mmdet.DiceLoss` reach across to mmdet
`cudnn_benchmark`	`True`	Lets cuDNN pick algorithms for a fixed input size. Fine here — every tile is 1024 × 1024
`mp_start_method`	`fork`	Ignored on Windows: mmengine applies it only on other systems, so it is inert on the machine most readers use
`dist_cfg`	`nccl`	Multi-GPU only. Unused in a single-GPU run
`load_from` / `resume`	`None` / `False`	Set by `--load-from` and `--resume` on the command line. Chapter 13 explains why the two are not interchangeable
`randomness`	commented out	So no seed is set, and two runs of one config do not agree exactly (chapter 1). Uncommenting it costs throughput and changes results slightly

6.9 What to change first, and how

After the folders are in place, these are the fields worth setting before a first run of your own. Everything in the last column is preferred: a command-line flag leaves the file — and therefore the published scores — untouched.

Want to	Field	Do it with
Point at your tiles	`data_root`	`--data-root ..\data\ghaf` on `train.py` and `test.py`
Train for less than 160,000 iterations	`max_iters`, and `end` in `param_scheduler`	A copied config. Both, or the decay curve is wrong

Want to	Field	Do it with
Validate and checkpoint more often	`val_interval`, and the checkpoint hook's `interval`	`--cfg-options train_cfg.val_interval=1000` — these are read at run time, so the flag works
Fit a smaller GPU	`batch_size`, in `train_dataloader`	`--cfg-options train_dataloader.batch_size=1`. It changes the result; say so when reporting
Start from a released checkpoint	`load_from`	`--load-from`, never `--resume` (chapter 15)
Make a run repeatable	`randomness`	Uncomment it in a copied config
Put logs elsewhere	`work_dir`	`--work-dir`

Making a variant properly

Copy the shared file, change the copy, and point a new model config at it. The six delivered configs go on meaning what they meant.

```
copy configs\_base_\ghaf.py configs\_base_\ghaf-new-site.py
copy configs\ghaf\fastvit-ma36_mask2former.py ^
configs\ghaf\fastvit-ma36_new-site.py
```

Then edit one line at the top of the new model config, so it reads `_base_ = ['../_base_/ghaf-new-site.py']`, and make your changes in `ghaf-new-site.py`. Confirm the result before training with it: `python -c "from mmengine.config import Config; print(Config.fromfile(r'configs\ghaf\fastvit-ma36_new-site.py').train_dataloader.batch_size)"` prints what the runner will actually use, which is not always what the file appears to say.

Checklist

- You know which of the two files owns the field you want to change.
- You have not edited anything under `configs/_base_/` in place.
- `max_iters` and `param_scheduler[0].end` still agree.
- A changed batch size or crop size is recorded wherever the results are reported.

CHAPTER

7 Training

Only worth doing if something is to be changed: six trained models are supplied. This chapter covers a run from scratch, what it writes, and the one setting that makes a run impossible to reproduce.

7.1 The run

```
cd /d D:\ghaf-project\code
python tools\train.py configs\ghaf\fastvit-ma36_mask2former.py ^
--data-root ..\data\ghaf ^
--init-weights ..\init-weights
```

`--init-weights` points at the ImageNet weights in the bundle, which is what lets a machine with no internet access start a run. Without it the backbones try to download their initialisation.

Setting	Value	Declared in
Iterations	160,000	`configs/_base_/ghaf.py`
Validation interval	every 3,500 iterations	same
Checkpoint interval	every 3,500 iterations, best on mIoU	same
Schedule	PolyLR, power 0.9	same
Batch size	2 train, 1 validation, 1 test	same

Setting	Value	Declared in
Optimiser (FastViT)	AdamW, lr 1e-4, weight decay 0.05, grad clip 0.01	`configs/ghaf/fastvit-ma36_mask2former.py`

Checkpoints and logs go to `work_dirs\<config-name>\`, relative to the working directory. An interrupted run continues with `--resume`, which restores the optimiser state and the iteration count. `--amp` enables mixed precision where the configuration supports it. How long a full run takes on the A5000 was never recorded.

> **NOTHING RECORDS THE SEED, SO NO RUN CAN BE REPEATED**
> The line `randomness = dict(seed=0, deterministic=True)` is commented out in `configs/_base_/ghaf.py`. mmengine therefore picks a seed at random and writes it into the run log — and nowhere else. Two runs of the same configuration on the same data will differ, and if `work_dirs\` is deleted the seed that produced a published number is gone for good. Keep the logs, or uncomment the line before starting anything you will need to defend.

Checklist

- `check_dataset.py --full` passed on the data first.
- `--init-weights` was passed, or the machine has internet access.
- `work_dirs\` is on a disk with room, and will not be deleted.
- The seed in the run log was copied somewhere it will survive.

CHAPTER 8

Evaluating a model

Scoring a model against the labelled test split reproduces the published figures. It is the strongest single check that an installation, a checkpoint and a dataset are all the ones they claim to be. Three minutes on a GPU.

```
cd /d D:\ghaf-project\code
set MODEL=..\models\fastvit-ma36_mask2former
python tools\test.py ^
%MODEL%\fastvit-ma36_mask2former.py ^
%MODEL%\best_mIoU_iter_3500.pth ^
--data-root ..\data\ghaf
```

The last line reports mIoU, mDice and mFscore, with a per-class table above it. For FastViT-MA36 expect **79.32** and **87.22**; the other five are in chapter 1. A departure larger than a rounding difference has two usual causes, and chapter 4 settles the second: a data root pointing somewhere unintended, or a checkpoint that is not the one it is taken for.

`--show-dir` writes prediction visualisations, and `--work-dir` places the log somewhere other than the default.

Checklist

- The reported mIoU matches the published figure for that model.
- `--data-root` pointed where you thought it did.
- If it did not match, chapter 4 check 2 was run before anything else.

CHAPTER

9 Predicting a labelled split, tile by tile

Scoring reduces a split to three numbers (chapter 8). The maps behind those numbers are what error analysis and figures are made from: they show where a model is wrong rather than by how much. This is also the smallest prediction the pipeline does, and the one to run first.

```
cd /d D:\ghaf-project\code
set MODEL=..\models\fastvit-ma36_mask2former
python tools\predict_split.py ^
%MODEL%\fastvit-ma36_mask2former.py ^
%MODEL%\best_mIoU_iter_3500.pth ^
--data-root ..\data\ghaf --split testing ^
--out-dir ..\output\predictions --save-probability
```

One mask per tile, encoded exactly as the ground truth is, so a prediction and its label can be subtracted directly. Run `--limit 20` first and open the result before committing to 767 tiles.

9.1 What it writes

```
predictions\
  masks\           one .tif per tile, 0 and 1
  probability\     float32 P(ghaf), with --save-probability
  summary.json    every tile, its canopy share, and the totals
```

A test tile carries a `.pgw` world file, so its prediction opens in QGIS in the right place (chapter 5). A training or validation tile carries none, and its prediction is an image without a position — correct, and not a fault.

Option	Effect
`--split`	`training`, `validation` or `testing`
`--limit 20`	Stop after twenty tiles
`--save-probability`	Keep the float32 confidence raster as well as the 0/1 mask
`--threshold`	The confidence at which a pixel becomes a crown. 0.5 by default, as everywhere else in the pipeline
`--batch-size`	Tiles per forward pass. 4 fits the workstation in chapter 3

The canopy fraction over the test split is 3.44 per cent. The delivered bundle already contains these 767 predictions, so this needs running only for a model other than FastViT-MA36, or after training one of your own.

Checklist

- `--limit 20` was run and looked at before the full split.
- `summary.json` reports the tile count the split holds.
- The canopy share is near 3.44 per cent for the test split.

CHAPTER

10 Mapping one orthomosaic

The main job. An orthomosaic goes in, crown polygons come out, and the image may be far larger than the memory of the machine. Start with the sample clip: it proves the whole chain in a couple of minutes rather than a couple of hours.

10.1 How it works, in one paragraph

The mosaic is read in 1024-pixel windows that overlap by 512. Each window is predicted independently, and the predictions are combined with Gaussian weights that fall off toward the window edge, so a pixel covered by four windows takes a weighted mean rather than whichever window happened to be written last. That is what leaves no seams. The accumulators are memory-mapped files rather than arrays, so peak memory does not grow with the mosaic — but disk does, at 9 bytes per source pixel.

10.2 The command

Work from `code\`, which keeps the paths short enough to read. The caret continues the line in the Command Prompt.

```
cd /d D:\ghaf-project\code
set MODEL=..\models\fastvit-ma36_mask2former
python -m ghaf.inference.large_image ^
%MODEL%\fastvit-ma36_mask2former.py ^
%MODEL%\best_mIoU_iter_3500.pth ^
..\samples\Kalba26_sample.tif ^
--out-mask ..\output\crowns.tif ^
--out-prob ..\output\probability.tif ^
--out-polygons ..\output\crowns.gpkg ^
--batch-size 4 --min-area 1
```

The three positional arguments are the configuration, the weights and the image, in that order. At least one output path is required, and `--out-polygons` additionally requires `--out-mask` or `--out-prob`, because the polygons are traced from a written raster rather than from memory. Passing polygons alone is refused by the argument parser, not discovered later.

```
INFO Kalba26_sample.tif: 8192 x 8192 px, 225 window(s) of 1024 px
     (overlap 512), batch 4, 0.6 GB scratch
Kalba26_sample: 100%|##############| 57/57 [00:40<00:00, 1.42batch/s]
INFO Created 27 records
INFO canopy: 650055 of 67108864 valid px (0.97%)
```

10.3 The three outputs

File	What it holds	What it is for
`crowns.gpkg`	The crowns as polygons, with `area_m2` per feature	Counting, measuring, joining to other layers. The primary result
`crowns.tif`	One band, 1 for crown and 0 for background, the size of the input	Overlaying, canopy cover, differencing against a labelled mask
`probability.tif`	One band of float32, the confidence per pixel from 0 to 1	Re-thresholding after the run, and seeing where the model was unsure

All three carry the CRS of the input, so they land in place with no manual positioning.

10.4 Whether the answer is plausible

Quantity	On the sample clip	If it is far from that
Canopy share	0.9687 per cent of valid pixels	`0.00` means nothing was found; above 50 per cent means everything was. Both point upstream — usually the wrong checkpoint, or bands that are not red, green, blue
Polygons	27 at the defaults, 16 with `--min-area 1`	A much larger count is single-pixel fragments, not trees
Crown area	2.4 to 112 m²	Hundreds of square metres means crowns merged, or the imagery is not near the 2.68 cm ground sampling distance the models were trained at

The polygon areas on the sample clip sum to 469 m², against 467 m² computed from the mask pixels. The two should agree to about that; a wide gap means the polygons and the raster came from different runs.

10.5 Options that change the answer

Option	Default	Effect
`--threshold`	0.5	The confidence at which a pixel becomes a crown. Higher gives fewer and more certain crowns
`--min-area`	0.0	Discards polygons below this many square metres. At the default the polygon layer matches the mask exactly, fragments included
`--batch-size`	1	Windows per forward pass. Raise it until VRAM is the limit; 4 was used for every timing here
`--tile` / `--overlap`	1024 / 512	**Do not change** `--tile`. It must match the crop size the model was trained at
`--bands`	1 2 3	Which bands are red, green and blue
`--scratch-dir`	system temp	Where the accumulators go. Point it at a large local disk
`--device`	`cuda:0`	`cpu` runs without a GPU

Defaults are the argparse defaults in `ghaf/inference/large_image.py`*, not recommendations.*

> **CHANGING THE TILE SIZE SUCCEEDS AND GIVES A WORSE ANSWER**
> The model was trained on 1024-pixel crops. Run it at 512 and it still runs, still writes all three outputs, and still reports a canopy percentage — one produced by a model looking at the imagery at the wrong scale. Nothing in the output records the tile size that produced it. If you change it, write it down beside the result.

10.6 The full survey mosaic

`Kalba26.tif` is 84,072 × 103,691 pixels, or 8.7 billion. At 9 bytes per pixel the run needs **78.5 GB** of scratch space and plans **33,128** windows. Both are arithmetic on the raster size, not measurements: the full mosaic has never been run end to end, and no timing for it exists. Free space is checked before the run starts rather than part way through, which is the one thing that will not surprise you.

10.7 Cutting a clip to try first

```
python tools\make_sample.py ..\samples\Kalba26.tif ^
--output ..\samples\my_sample.tif --size 8192 --origin 30000 40000
```

`--size` takes one number for a square or two for a rectangle. `--origin` is the top-left corner in pixels; omitted, the clip is taken from the centre. The tool reports what share of the clip is imagery rather than the transparent border around a survey, so a window that landed on nothing is obvious before the model runs.

Checklist

- The sample clip ran, and reported about 0.97 per cent canopy.
- You passed `--out-mask` or `--out-prob` alongside `--out-polygons`.
- You did not change `--tile`.
- `--scratch-dir` points at a disk with room for 9 bytes per pixel.

CHAPTER

11 Mapping every image in a folder

A survey usually arrives as many images rather than one mosaic. This maps all of them in a single run and writes one GeoPackage of crowns per image. Each image is windowed exactly as a full mosaic is, so they need not share a size.

```
cd /d D:\ghaf-project\code
set MODEL=..\models\fastvit-ma36_mask2former
python tools\predict_folder.py ^
%MODEL%\fastvit-ma36_mask2former.py ^
%MODEL%\best_mIoU_iter_3500.pth ^
D:\my-images --out-dir ..\output\folder ^
--batch-size 4 --min-area 1
```

11.1 What it writes

```
folder\
  polygons\      one .gpkg of crowns per image, with area_m2
  masks\         the 0/1 raster, only with --save-mask
  probability\   the confidence raster, only with --save-probability
  summary.json   every image, its canopy share, and any failures
```

Crowns are the output and the rasters are opt-in. A GeoPackage is a few hundred kilobytes where the mask it was traced from is hundreds of megabytes, and counting, measuring and drawing all work from the polygons. The mask is still produced, because the polygons are traced from it, but it goes to the scratch directory and is deleted when that image finishes. `--save-mask` keeps it.

Subfolders in the input are reproduced in the output, so two images of the same name in different folders cannot overwrite one another. The output folder is excluded from a recursive listing, so a second run does not predict the first run's own rasters.

11.2 Options for a long run

Option	Effect
`--limit 3`	Stop after three images. Do this first on a large folder: it proves the settings before several hundred images are committed to
`--recursive`	Include images in subfolders
`--pattern "*_rgb.tif"`	Only files whose name matches. Useful where a folder mixes imagery with elevation models
`--skip-existing`	Resume an interrupted run instead of repeating finished images
`--save-mask`, `--save-probability`	Keep the rasters as well
`--min-area 1`	Discard crowns under one square metre

READ SUMMARY.JSON AFTER ANY LONG RUN

One unreadable file does not stop the batch. The image is reported, the run continues, and the failure is recorded in `summary.json` with the exception that caused it. The exit status is non-zero if anything failed — but an operator watching a progress bar scroll past will not see it, and a folder of 400 images that produced 397 GeoPackages looks complete in a file browser.

Checklist

- `--limit 3` was run first, and the crowns looked right in QGIS.

- `summary.json` reports `0 failed`, or you have read the failures.
- The canopy share per image is in the range chapter 10 gives.

CHAPTER

12 Reviewing the results

Opening the outputs, and the numbers to check them against. Every file the pipeline writes carries the coordinate system of the image it came from, so nothing needs positioning by hand.

12.1 The crowns in QGIS

- **Layer > Add Layer > Add Vector Layer**, choose `crowns.gpkg`, **Add**.
- Add the imagery through **Add Raster Layer** and drag it below the crowns in the Layers panel.
- Right-click the crowns > **Properties > Symbology**. Set the fill to *No brush* and pick a bright outline, so the imagery stays visible inside each crown.
- Right-click > **Open Attribute Table**. One row per crown, with `area_m2`. The row count is the crown count.
- **Vector > Analysis Tools > Basic Statistics for Fields** on `area_m2` gives the count, sum, mean, minimum and maximum.

12.2 The rasters in QGIS

Both open through **Add Raster Layer**. The mask appears black, because its values are 0 and 1 while QGIS stretches the display over 0 to 255; set the render type to **Paletted/Unique values** and click **Classify** to get one colour per class. For the probability raster use **Singleband pseudocolor** from 0 to 1.

12.3 Figures to check against

Quantity	Value	Where it came from
Canopy share, sample clip	0.9687 per cent of valid pixels	Measured: 650,055 of 67,108,864 px
Canopy share, test split	3.44 per cent over 767 tiles	Measured by `predict_split.py`
Crowns, sample clip	27 raw, 16 after `--min-area 1`	Measured
Crown area, sample clip	2.4 to 112 m²	Measured
Total crown area, sample clip	469 m² from polygons, 467 m² from pixels	Measured
Ground sampling distance	2.68 cm per pixel	From the transform of the sample clip
Tile ground size	27.44 m square, 0.075 ha	Derived: 1024 × 2.68 cm

All measured on the delivered models and the sample clip. None of them is a target — they are what this imagery produced.

Checklist

- The crowns sit on the imagery without being moved.
- The canopy share is within an order of magnitude of the table above.
- The crown count came from the attribute table, not from an estimate.

CHAPTER

13 Adapting to a new site

Fine-tuning from a released checkpoint costs far less than training from scratch and usually scores better. It is also the operation most likely to produce a model that looks fine and is worse than the one it started from.

13.1 Prepare and check the new tiles

Same layout as chapter 5: 1024 × 1024 PNG pairs, masks containing only 0 and 1, `training` and `validation` folders. Check them before the run, not after.

```
python tools\check_dataset.py D:\ghaf-project\data\new-site --full
```

13.2 The run

```
set MODEL=..\models\fastvit-ma36_mask2former
python tools\train.py configs\ghaf\fastvit-ma36_mask2former.py ^
--data-root ..\data\new-site ^
--load-from %MODEL%\best_mIoU_iter_3500.pth ^
--cfg-options train_cfg.max_iters=4000 ^
optim_wrapper.optimizer.lr=1e-5
```

Argument	What it does
`--load-from`	Takes the weights and starts a fresh schedule. This is what fine-tuning means
`--resume`	Continues an interrupted run of your own, keeping the optimiser state and iteration count. A different operation entirely, and confusing the two silently restarts a schedule
`max_iters=4000`	A short schedule. The full 160,000 would overwrite what the model already knows
`lr=1e-5`	A tenth of the training rate, so the model adapts rather than forgets

A FINE-TUNED MODEL CAN SCORE WELL AND BE WORSE

Fine-tuning on a small site teaches the model that site. Score the result on the **original** test split as well as the new one before replacing anything: a model that scores 0.88 on the new site and has lost four points on `testing/ghaf26` is not an improvement, and nothing in the training log will say so. No fine-tuning run has been performed on this project, so there is no measured example of how far it drifts.

Score the result exactly as in chapter 8, with `--data-root` at the new site and the checkpoint at the new `work_dirs\...\best_mIoU_iter_*.pth`, then again with `--data-root ..\data\ghaf` to see what it cost.

Checklist

- The new tiles passed `check_dataset.py --full`.
- `--load-from` was used, not `--resume`.
- The result was scored on both the new site and the original test split.
- The original checkpoint is still where it was; nothing overwrote it.

CHAPTER

14 Handing the system on

How the bundle in chapter 1 is assembled, and how a recipient checks what they were sent. Two programs, both of which verify rather than trust.

14.1 Packaging the models

```
python tools\export_release.py ^
--checkpoints D:\ghaf-project\checkpoints ^
--output D:\ghaf-release
```

One self-contained folder per model: a resolved configuration beside its weights and a `metadata.json`. The configuration is flattened, so it carries the whole recipe and does not need `configs/_base_/`. Every checkpoint is verified against the SHA-256 in `ghaf/release.py` before the copy and again after, which means the command accepts only the six released checkpoints and a bundle cannot carry a silently corrupted file. `--dry-run` verifies and reports without writing; `--only <key>` limits it to one model.

14.2 Assembling the whole bundle

```
python tools\build_handover.py --output D:\ghaf-project ^
--code D:\ghaf-public --checkpoints D:\checkpoints ^
--init-weights D:\init-weights --data D:\tiles\ghaf ^
--samples D:\samples --predictions D:\predictions
```

Inside a git checkout, `--code` copies exactly what `git ls-files` reports and records the commit in `MANIFEST.json`; files that are not tracked are counted and reported rather than copied. `--data` copies the three named splits and nothing else, which on the run that produced this bundle meant 19,583 files copied and 49,490 left behind. `--dry-run` reports the sizes without writing, and is worth running first: the first attempt on this project reported 103 GB because the tile tree was a working directory.

14.3 What a recipient runs

The check that matters is chapter 4 check 2, pointed at the models folder they were sent. It confirms every digest, loads every checkpoint into the model built from the configuration, and runs a prediction through each.

```
python tools\smoke_test.py --checkpoints ..\models
python tools\check_dataset.py ..\data\ghaf --sample 0
```

`--sample 0` checks pairing without opening a file, which is what makes it usable over a network share; a full check of 8,641 tiles across a mounted drive takes long enough that it is usually abandoned half way, which is worse than not starting it.

Checklist

- `build_handover.py --dry-run` was read before the real run.
- `MANIFEST.json` records the commit the code came from.
- The recipient ran `smoke_test.py --checkpoints` and got six `ok`.
- The full orthomosaic was handled separately; it is not in the bundle.

CHAPTER

15 Error catalogue

Keyed on what appears on screen. Find the message, read across. Everything here has actually happened on this project or is raised explicitly by its code; nothing is a hypothetical.

15.1 How to read a failure

Python prints a record of what it was doing and then, on the last line, what went wrong. The last line is the error; everything above it is context. The word before the colon is the category. Copy the text before running anything else — closing the window loses it, and an error nobody can quote is an error nobody can diagnose.

Three questions resolve most of it before any searching. Does the prompt read `(ghaf)`? Does `python -c "import sys; print(sys.executable)"` print a path containing `envs\ghaf`? Are you in `code\`?

15.2 Messages

What you see	What it means, and what to do
`No module named mmengine`, `pytest` or `ghaf`	A different Python is running than the one the packages went into. `conda activate ghaf`, then check `sys.executable`. Always start commands with `python -m`
`mmseg ... is not installed in conda environment "X"`	Wrong environment, or the stack was never installed on this machine. The message names the interpreter it asked
`No module named ftfy`	mmsegmentation 1.2.2 imports a tokenizer needing it, though its own metadata does not declare it. `python -m pip install ftfy regex`
`RuntimeError: Numpy is not available`	NumPy 2 beside a PyTorch built for 1.x. `python -m pip install "numpy<2"`
`opencv-python ... requires numpy>=2` while installing	Not an error. The wheel is built against the NumPy 2 headers and works with either
`Attempting uninstall: opencv-python`, replacing it with 4.11.0.86	Correct, and required. `numpy<2` is pinned for PyTorch 1.12.1, and 4.11.0.86 is the newest OpenCV built against NumPy 1.x. Nothing here imports OpenCV directly; mmengine, which does, accepts any version from 3 upwards
`CUDA out of memory`	Lower `--batch-size` to 2 or 1, close other GPU programs, or use `--device cpu`. Do not lower `--overlap` to save memory; it changes the result
`not enough scratch space`	The drive holding temporary files is too small. `--scratch-dir` at a larger disk. An image needs 9 bytes per pixel
`nothing to do: pass at least one of --out-prob, --out-mask, --out-polygons`	The parser refusing a run with no output. Add one
`--out-polygons needs --out-mask or --out-prob to vectorise from`	Polygons are traced from a written raster. Add `--out-mask`
`FileNotFoundError` naming a path	A typo, a missing `..\`, or a path with spaces that needs quotes
`not a directory: ...` naming `D:\ghaf-project`	The example path from this manual, run as typed. `D:\ghaf-project` stands for wherever the bundle was unpacked on this machine (chapter 3); no program looks for that name. Both `smoke_test.py` and `check_dataset.py` refuse a folder that is not there before doing any work
`not a directory` from `check_dataset.py` on a path that exists	The dataset root given one level out. The argument is the folder that holds `training\`, `validation\` and `testing\`
`the dataset is not where this run was told to look`	`train.py` or `test.py` without `--data-root`. The config carries `data/ghaf`, which is read relative to `code\` and is not where the data is. The message names the folder it wanted and the directory it resolved against
`no ...pth under ...` from `smoke_test.py`	The folder is there and the checkpoints are not. Usually the models folder one level out, or a partial copy
`'ls' is not recognized as an internal or external command`	A Unix command in the Command Prompt. `dir` lists a folder, `type` prints a file, `cd /d` changes drive and folder together

What you see	What it means, and what to do
`The system cannot find the path specified`, printed twice	The path contains `&`, which the Command Prompt read as two commands. Quote the whole path
`The process cannot access the file because it is being used by another process`	QGIS or another program holds the file open. When deleting a folder, first `cd` out of it: a terminal inside a folder holds it
`NO TILES` from `check_dataset.py`	The folders exist and hold no PNG or TIF. The row names what it found instead; the tiles are usually one level deeper
`SHA-256 mismatch` from `smoke_test.py`	That checkpoint is not the released file. The copy is damaged; copy it again from the original
A failing test on a machine where the imports all work	Send the failure on; do not work around it. A test can assert something that is true only on the operating system it was written on — a sort order, a path separator — and that is a fault in the test, not in this installation. Copy the `assert` line, the expected value and the actual one
`0.00% ghaf` from `smoke_test.py`	Correct. That forward pass runs on a blank synthetic tile
`KeyboardInterrupt`	Ctrl+C, or the window closed. Nothing is damaged. Long runs resume with `--skip-existing` or `--resume`
Screens of `UserWarning` about `__floordiv__` or `meshgrid`	Deprecation notices from inside PyTorch and mmsegmentation. They appear on a correct run

15.3 Failures that produce a result

The messages above announce themselves. These do not: each one finishes cleanly, writes output, and reports a number that is wrong.

What you see	What it means, and what to do
`0.00%` canopy on real imagery	The run found nothing and said so calmly. Usually the wrong checkpoint, or bands that are not red, green, blue. Check both, then try `--bands`
Canopy above 50 per cent	The opposite failure, same causes
A crown count far above the range in chapter 10	Single-pixel fragments counted as trees. `--min-area 1` removed 11 of 27 on the sample clip
A model that trains to a high mIoU and predicts nothing	Masks encoded 0 and 255 rather than 0 and 1. Background alone is about 96 per cent of the pixels, so predicting nothing scores well. `check_dataset.py` is the only thing that catches it
Numbers that differ from a previous run of the same command	No seed is set (chapter 7). Also check the tile size, which nothing records in the output
A folder run that produced fewer files than there were images	One or more images failed and the batch continued. `summary.json` names them; the exit status was non-zero

15.4 Searching for anything else

Take the last line of the error and strip what is specific to your machine — user name, drive letters, file names, long numbers. None of it appears in anyone else's error. Add the library that raised it. Search that. Errors raised inside the framework belong on the mmsegmentation issue tracker, closed issues included, since a closed issue usually holds the fix; the address is in chapter 16.

Two rules when judging what you find. Check the versions: an answer written for mmsegmentation 0.x does not apply to 1.2.2. And do not upgrade a package on general advice — the versions in chapter 3 were chosen to work together and to match the trained weights, and `pip install --upgrade` will replace one and break four. Change one thing, then re-run chapter 4.

If an installation reaches a state that cannot be explained, delete the environment and rebuild it from chapter 3. Twenty minutes, and it touches no data: `conda deactivate`, `conda env remove -n ghaf`, then step 1.

15.5 Reporting it to somebody else

- The exact command, copied rather than retyped.

- The complete output, as text, from the command to the last line.
- What you expected instead.
- The output of `python -c "import sys; print(sys.executable)"`.
- The `torch`, `mmcv`, `mmsegmentation`, `mmdet` and `numpy` lines from `python -m pip list`.
- Whether chapter 4 passes now, and whether it ever did on this machine.

CHAPTER

16 Reference

Every routine command in one place, the released models by digest, and what was never established. The quick reference sheet carries a shorter version of the first table.

16.1 Commands

All assume `conda activate ghaf` and `cd /d D:\ghaf-project\code`, with `MODEL` set as in chapter 10.

```
python -m pytest tests\ -q
python tools\smoke_test.py --checkpoints ..\models
python tools\check_dataset.py ..\data\ghaf --sample 0
python tools\check_dataset.py ..\data\ghaf --full

python -m ghaf.inference.large_image %MODEL%\CONFIG %MODEL%\WEIGHTS ^
image.tif --out-mask crowns.tif --out-polygons crowns.gpkg ^
--batch-size 4 --min-area 1

python tools\predict_folder.py %MODEL%\CONFIG %MODEL%\WEIGHTS ^
D:\my-images --out-dir ..\output\folder --batch-size 4

python tools\make_sample.py mosaic.tif --output clip.tif --size 8192
python tools\test.py %MODEL%\CONFIG %MODEL%\WEIGHTS ^
--data-root ..\data\ghaf
python tools\train.py configs\ghaf\CONFIG --data-root ..\data\ghaf
```

16.2 The released models

Model	Checkpoint	Bytes	SHA-256, first 16
`fastvit-ma36_mask2former`	`best_mIoU_iter_3500.pth`	252,650,755	`f26cd5257b55058f`
`poolformer-s36_fpn`	`iter_10200.pth`	416,437,419	`59683e3788548494`
`dpn98_fpn`	`best_mIoU_iter_14000.pth`	263,385,401	`e292f2262f132f57`
`convnext-small_upernet`	`iter_14000.pth`	983,511,196	`8435410e25140545`
`resnet-50_mask2former`	`best_mIoU_iter_38500.pth`	196,545,149	`5a79f618902be032`
`efficientnet-b3_fpn`	`iter_6800.pth`	108,104,299	`9d6131e21a1ec5f3`

Identify a checkpoint by its digest, not by its filename. The full digests are in `ghaf/release.py` and in each `metadata.json`.

16.3 What was never established

Stated so that nobody has to work out whether a number is missing or merely not repeated here. None of the following was measured, and this manual does not estimate them.

- Wall-clock time for a full-mosaic inference run; the mosaic has never been run end to end.
- Wall-clock time for training or for fine-tuning, on any hardware.
- Validation scores for the five models other than FastViT-MA36.
- Inference throughput per model.

- GPU memory actually consumed at batch size 4, which is why chapter 10 gives a starting point rather than a recommendation.
- The date each released checkpoint was trained, and which run in `work_dirs` produced it; the logs were not inspected.
- Any fine-tuning result on a second site, and therefore how far a fine-tuned model drifts from the original.
- How often touching crowns merge into one polygon on this imagery.

16.4 Where to look when it is not here

Source	For
`docs/handover/FACTS.yml`	Every value this manual prints, with its provenance
github.com/open-mmlab/mmsegmentation	Errors raised inside the framework. Search closed issues too
`python tools/<program>.py --help`	What an option is called and what it does
`docs/AREA_WIDE_INFERENCE.md`	How a mosaic is windowed and blended, in more detail than chapter 10
`docs/MODEL_ZOO.md`	Per-class scores and training settings

The authority for every value in this manual is `docs/handover/FACTS.yml`. Each entry there carries its provenance: read from a file in the repository, measured by a command that was run, derived arithmetically, or marked as never established. Where this manual and that file could disagree, the file wins.